

Builders Stream

Software Development Documentation

A multi-tenant SaaS platform for construction & renovation project management

Version 1.0 · 2026-07-07

Table of Contents

Builders Stream — Product Requirements Document (PRD)	6
1. Executive Summary	6
2. Problem Statement	6
3. Target Users / Personas	6
4. Goals & Success Metrics	7
5. Product Scope	7
5.1 In Scope (Full Vision)	7
5.2 Out of Scope (v1)	7
6. MVP Definition	7
7. Assumptions & Constraints	8
8. Open Questions	8
Builders Stream — Functional Requirements Specification	9
1. Users & Company Management (P0)	9
Core Entities	9
User Stories	9
Roles (RBAC)	9
Business Rules	10
2. CRM (P1)	10
Core Entities	10
User Stories	10
Business Rules	10
3. Project Management (P1)	11
Core Entities	11
User Stories	11
Business Rules	11
4. Estimation Engine (P1/P2 — MVP)	11
Core Entities	11
User Stories	12
Business Rules	12
5. Accounting & Billing (P3 — Post-MVP)	12
Core Entities	12
User Stories	13
Business Rules	13
6. External Integrations — QuickBooks / FreshBooks (P4 — Post-MVP)	13
User Stories	13

Business Rules	13
7. Compliance Tracking (Cross-Cutting)	13
User Stories	13
Business Rules	14
Builders Stream — Technical Architecture	15
1. Architecture Pattern	15
2. Technology Stack	15
3. Modular Domain Decomposition	15
4. Cross-Module Communication: Internal Event Bus	16
5. Multi-Tenancy & Data Isolation	17
5.1 Tenant Model	17
5.2 Enforcement: PostgreSQL Row-Level Security (RLS)	17
5.3 Tenant Context Propagation (Frontend → Backend)	18
6. Estimation Calculation Rules	18
7. Asynchronous Processing	18
8. Deployment Topology (Self-Hosted)	19
9. Open Architectural Question: Mobile/Offline	19
Builders Stream — Database Schema Design	21
1. Entity-Relationship Overview	21
2. Users & Company Management (P0)	21
3. CRM (P1)	22
4. Project Management (P1)	23
5. Estimation Engine (P1/P2)	24
6. E-Signatures & Compliance (Cross-Cutting)	25
7. Post-MVP Tables (Phase 3–4, Structure Only)	26
8. Audit Log (Cross-Cutting, P0)	26
9. Indexing & RLS Notes	27
Builders Stream — API Specification	28
1. Conventions	28
2. Users & Company Management	28
3. CRM	29
4. Project Management	29
5. Estimation Engine	30
6. Accounting & Billing (Post-MVP)	30
7. External Integrations (Post-MVP)	31
8. Compliance	31

9. Webhooks (Inbound)	31
10. Security Requirements on Every Endpoint.....	31
Builders Stream — Non-Functional Requirements	33
1. Performance.....	33
2. Scalability	33
3. Availability	33
4. Backup & Disaster Recovery	33
5. Observability.....	34
6. Self-Hosted Infrastructure Requirements	34
7. Browser & Device Support.....	34
8. Data Retention	34
Builders Stream — Security & Compliance Plan	35
1. Authentication.....	35
2. Authorization (RBAC Matrix).....	35
3. Tenant Isolation	35
4. Encryption	36
5. Audit Logging	36
6. E-Signature Workflow	36
7. Data Retention	36
8. Compliance Tracking Feature (Insurance/License Expiry)	37
9. Incident Response (Outline)	37
10. Third-Party Data Processors.....	37
Builders Stream — Pricing & Subscription Model.....	38
1. Model: Tiered Plans by Feature + Seat Count	38
2. Tier Definitions	38
3. Feature Gating Table.....	38
4. Stripe Implementation Mapping	39
5. Trial & Conversion	39
6. Upgrade / Downgrade Rules	39
Builders Stream — Roadmap & Implementation Plan	40
Phase 0 — Foundation (Pre-MVP)	40
Phase 1 — CRM & Project Management (MVP core).....	40
Phase 2 — Estimation Engine + E-Signature (MVP completion)	41
Phase 3 — Compliance Tracking + Accounting/Billing (Post-MVP)	41
Phase 4 — External Integrations (Post-MVP)	41
Phase 5 — Open Items (Scope TBD, Not Yet Scheduled).....	41

Milestone Summary42

Builders Stream — Test Strategy43

1. Test Pyramid43

2. Tenant Isolation Testing (Release Gate)43

3. Estimation Engine Testing44

4. State Machine Testing44

5. E-Signature & Compliance Testing44

6. Third-Party Integration Testing44

7. Performance/Load Testing44

8. CI Gates44

Builders Stream — Product Requirements Document (PRD)

Version: 1.0

Date: 2026-07-07

Status: Draft for review

1. Executive Summary

Builders Stream is a multi-tenant SaaS platform for small/medium building construction and renovation companies. It unifies CRM, project management, cost estimation, accounting/billing, and company administration into a single system, targeting 1,000+ paying company subscribers on a self-hosted infrastructure.

2. Problem Statement

Small and medium construction/renovation contractors currently run their business across a fragmented set of disconnected tools: spreadsheets for estimating, a separate CRM (or none), email/text for client communication, and a bolt-on accounting package for invoicing. This fragmentation ("the fragmentation tax") causes:

- Lost or duplicated data entry between tools (a lead's details get re-typed into an estimate, then again into an invoice).
- No single source of truth for a project's financial state (estimated vs. actual cost).
- Manual, error-prone handoffs between sales (CRM), execution (project management), and billing.
- Difficulty tracking compliance obligations (subcontractor insurance, licenses, signed change orders) until a problem occurs.

3. Target Users / Personas

Persona	Role	Primary Needs
Company Owner / Admin	Runs the company, owns the tenant	Visibility across all projects, financials, and branches; user/role management.
Project Manager	Manages day-to-day project execution	Task scheduling, document access, crew coordination, change order tracking.
Field Crew	On-site labor/subcontractors	Task lists, daily log submission, photo uploads.
Accountant / Bookkeeper	Managing financial records	Invoicing, expense tracking, QuickBooks/FreshBooks sync, financial reporting.
Client (external)	The homeowner/business paying for the project	Read-only progress visibility, estimate/change-order approval (e-signature).

4. Goals & Success Metrics

Goal	Metric
Reach commercial scale	1,000 paying company (tenant) subscribers.
Strict tenant data isolation	Zero cross-tenant data leaks, verified by automated RLS isolation tests (see Test Strategy).
Reduce fragmentation	A lead can move from "New" through "Won" → active Project → approved Estimate → Invoice without re-entering data.
Operational reliability	99.5%+ uptime on self-hosted infrastructure (see NFRs).

Note: precise adoption/revenue targets and conversion benchmarks are not yet defined and should be set once pricing (see [Pricing & Subscription Model](#)) is finalized against real market feedback — this PRD intentionally does not invent numbers that haven't been validated.

5. Product Scope

5.1 In Scope (Full Vision)

1. Users & Company Management (multi-tenant, nested company hierarchy, RBAC)
2. CRM (lead pipeline, client communication history)
3. Project Management (project lifecycle, tasks, documents, daily logs)
4. Estimation Engine (cost catalog, markup profiles, quote generation, e-signature approval)
5. Accounting & Billing (Stripe subscription billing, project invoicing)
6. External Integrations (QuickBooks / FreshBooks sync)
7. Compliance Tracking (subcontractor insurance/license expiry, signed change orders)

5.2 Out of Scope (v1)

- Native mobile apps (responsive web only; offline/PWA support is an open question — see Section 7).
- Payroll processing.
- Estimating via AI-driven blueprint takeoff (manual/assisted quantity entry only in v1).
- Multi-currency / multi-language support.

6. MVP Definition

The MVP is the minimum set of modules needed for a construction company to run its core sales-to-execution workflow inside Builders Stream:

- **Users & Company Management** (P0 — foundation, required by everything else)
- **CRM** (P1)
- **Project Management** (P1)
- **Estimation Engine**, including e-signature approval on estimates (P1/P2)

Accounting/Billing, QuickBooks/FreshBooks integration, and standalone insurance/license expiry tracking are explicitly **post-MVP** (see [Roadmap](#) for phasing and rationale).

7. Assumptions & Constraints

- **Team:** Solo developer, no hard external launch deadline — scope and pacing favor incremental delivery over a fixed date.
- **Hosting:** Self-hosted on the developer's own Proxmox/Dell PowerEdge infrastructure, not a managed cloud platform. This shapes the NFRs (Section 6) and deployment architecture.
- **Architecture:** Decoupled frontend/backend (Next.js + FastAPI) was chosen deliberately over a unified Next.js-only stack to preserve technology independence and support future non-web clients (mobile, integrations).
- **Offline/mobile field use:** Not yet decided. Flagged as an open question — if required later, it affects the Project Management module's sync architecture (see [Technical Architecture](#), Section 8).

8. Open Questions

#	Question	Owner	Status
1	Does field crew usage require offline-capable mobile/PWA support?	Product	Open
2	Final dollar pricing for Starter/Pro/Enterprise tiers	Product	Open — see Pricing Model
3	Target adoption/revenue metrics for the 1,000-subscriber goal	Product	Open

Builders Stream — Functional Requirements Specification

Version: 1.0

Date: 2026-07-07

Related: [PRD](#) · [Technical Architecture](#) · [Database Schema](#)

Each module lists its core entities, user stories with acceptance criteria, and business rules. Modules are tagged with their implementation priority from the [Roadmap](#): **P0** (foundation), **P1–P2** (MVP), **P3–P4** (post-MVP).

1. Users & Company Management (P0)

Core Entities

Company (with optional `parent_id` for nested branches), User, CompanyUser (role assignment), Invitation.

User Stories

US-1.1 — As a company owner, I can create a company account and become its first Admin.

- Acceptance: A new Company row is created; the creating user is assigned the `admin` role via CompanyUser.

US-1.2 — As an Admin, I can invite additional users to my company with a specific role.

- Acceptance: An invitation email is sent; accepting it creates a CompanyUser row with the assigned role; invitations expire after 7 days.

US-1.3 — As an Admin of a parent company, I can create child branch companies and view aggregated data across all of them.

- Acceptance: Child companies inherit no data by default but are visible to parent-company admins per the recursive RLS policy (see [Technical Architecture](#), Section 5).

US-1.4 — As any authenticated user, I can only see and act on data belonging to my company (and its children, if I have parent-level access).

- Acceptance: Verified by automated RLS isolation tests (see [Test Strategy](#), Section 2).

Roles (RBAC)

Role	Scope
Admin	Full CRUD across all modules within their company (and children, if parent).
Project Manager	Full CRUD on CRM, Projects, Tasks, Estimates within their company. No billing/subscription access.
Field Crew	Read assigned tasks; create Daily Logs and upload photos. No access to budgets or client contact financial data.

Role	Scope
Accountant	Full access to Billing/Accounting module; read-only elsewhere.
Client (external)	Read-only sanitized project dashboard; can approve/reject Estimates and Change Orders via e-signature.

Full permission matrix is defined in [Security & Compliance](#), Section 2.

Business Rules

- A user may belong to more than one company (e.g., a subcontractor working with multiple clients), but each membership has its own role.
- Deactivating a company (`is_active = false`) cascades to block login for all its users but does not delete data.

2. CRM (P1)

Core Entities

Lead, Communication Log (notes/emails/calls linked to a Lead).

User Stories

US-2.1 — As a Project Manager, I can create a new Lead with contact info, project type, and an estimated value.

- Acceptance: Lead is created with status `new`; validation matches the schema in [API Specification](#), Section 2.

US-2.2 — As a Project Manager, I can move a Lead through the pipeline: New → Contacted → Estimating → Qualified → Won/Lost.

- Acceptance: Status transitions are logged with a timestamp and the acting user.

US-2.3 — As a Project Manager, when I mark a Lead "Won," a Project is automatically drafted with the client details carried over.

- Acceptance: A `PROJECT_DRAFTED` event fires; a new Project row is created in `draft` status referencing the originating Lead.

US-2.4 — As any CRM user, I can log a communication (call, email, note) against a Lead and see a chronological history.

- Acceptance: Communication Log entries are immutable once saved (edits create a new entry, not an overwrite).

Business Rules

- A Lead cannot be deleted once it has an associated Estimate or Project; it can only be marked "Lost."
- Lead status "Estimating" is the trigger point that allows an Estimate to be created against it (see Section 4).

3. Project Management (P1)

Core Entities

Project, Phase/Milestone, Task, Document, Daily Log.

User Stories

US-3.1 — As a Project Manager, I can define Phases (e.g., "Site Prep," "Foundation," "Framing") and add Tasks within each Phase.

- Acceptance: Tasks require a name, due date, and assignee; they belong to exactly one Phase.

US-3.2 — As a Project Manager, I can advance a Project through its lifecycle: Draft → Pre-Construction → Active → Suspended → Completed → Archived.

- Acceptance: Illegal transitions (e.g., Draft → Completed) are rejected by the backend state machine.

US-3.3 — As Field Crew, I can view my assigned Tasks and submit a Daily Log (weather, progress notes, photos) for a Project.

- Acceptance: Daily Logs are timestamped, author-attributed, and immutable once submitted.

US-3.4 — As a Project Manager, I can upload and organize Documents (blueprints, permits, inspection reports) against a Project.

- Acceptance: Documents support versioning; the most recent version is shown by default with prior versions accessible.

US-3.5 — As a Client, I can view a read-only dashboard showing my project's progress, milestones, and approved photos, without seeing budget or internal task detail.

- Acceptance: Client-facing view excludes financial fields (verified by RBAC scoping — see [Security & Compliance](#)).

US-3.6 — As a Project Manager, I can create a Change Order against an active Project and route it to the Client for e-signature approval.

- Acceptance: Change Order requires a description, cost delta, and schedule impact; it is not binding until the client's e-signature is captured (see Section 4 and [Security & Compliance](#), Section 6).

Business Rules

- Hierarchical visibility: a user with parent-branch access sees Projects across all child branches; a child-branch user sees only their own branch's Projects.
- A Project cannot move to "Completed" while it has open (non-approved) Change Orders.

4. Estimation Engine (P1/P2 — MVP)

Core Entities

Estimate (Quote), Cost Catalog Item, Line Item, Markup Profile.

User Stories

US-4.1 — As a Project Manager, I can create a blank Estimate against a Lead or Project and select a Markup Profile.

- Acceptance: Estimate starts with zero line items and a status of **draft**.

US-4.2 — As a Project Manager, I can add Line Items to an Estimate by selecting Cost Catalog items and specifying quantities.

- Acceptance: Each Line Item's total is computed server-side as **quantity × unit_rate**; client-submitted totals are never trusted.

US-4.3 — As a Project Manager, I can trigger a recalculation that applies category subtotals, overhead markup, profit margin, and tax in a fixed order (see [Technical Architecture](#), Section 6) to produce a final total.

- Acceptance: All monetary math uses fixed-point decimal arithmetic; no floating-point rounding errors.

US-4.4 — As a Project Manager, I can export an Estimate as a branded PDF proposal and send it to the Client.

- Acceptance: PDF generation is asynchronous (background job); the requester receives a **202 Accepted** and is notified when the PDF is ready.

US-4.5 — As a Client, I can review an emailed Estimate and approve it with an e-signature, or reject it with a reason.

- Acceptance: Approval captures a timestamp, IP address, and signature artifact sufficient to satisfy the E-SIGN Act's intent-to-sign requirement (see [Security & Compliance](#), Section 6).

US-4.6 — As an Admin, I can define company-wide (or branch-specific) Cost Catalog items and Markup Profiles, with child branches able to override inherited values.

- Acceptance: A child branch's local override takes precedence over the parent's catalog entry for that item; the parent catalog itself is unaffected.

Business Rules

- **Historical immutability:** once an Estimate is Approved (client-signed) or marked Won, its line items are snapshotted and become immutable, even if the underlying Cost Catalog price later changes.
- An Estimate can only be created against a Lead in "Estimating" status or an existing Project.
- Approving an Estimate emits an **ESTIMATE_APPROVED** event consumed by the Accounting/Billing module (Section 5) to draft the initial invoice.

5. Accounting & Billing (P3 — Post-MVP)

Core Entities

Subscription (company's Builders Stream plan), Invoice, Payment, Expense.

User Stories

US-5.1 — As an Admin, I can view and manage my company's Builders Stream subscription (plan tier, seats, billing history) via a Stripe-hosted customer portal.

US-5.2 — As an Accountant, when a Project Estimate is approved, a draft client-facing Invoice is automatically generated for the deposit amount.

US-5.3 — As an Accountant, I can record Expenses against a Project and see running actual-vs-estimated cost.

US-5.4 — As an Accountant, I can view a financial report showing project profitability, outstanding invoices, and tax liability across the company.

Business Rules

- Builders Stream subscription billing (what the contractor pays to use the platform) and client-facing Project invoicing (what the contractor bills their own customer) are separate financial flows and must not be conflated in the data model.
- Financial records are never overwritten; corrections are made via new, linked entries (append-only ledger discipline).

6. External Integrations — QuickBooks / FreshBooks (P4 — Post-MVP)

User Stories

US-6.1 — As an Accountant, I can connect my company's QuickBooks or FreshBooks account via OAuth.

US-6.2 — As an Accountant, when an Invoice or Expense is created/updated in Builders Stream, it syncs to the connected accounting platform asynchronously, with retry on transient failure and a visible sync-status indicator.

Business Rules

- Sync failures must not block core Builders Stream workflows — the sync runs as a background task, decoupled from the request/response cycle (see [Technical Architecture](#), Section 4).

7. Compliance Tracking (Cross-Cutting)

Spans Users & Company (subcontractor/vendor records), Project Management (change order e-signatures), and Estimation (estimate e-signatures).

User Stories

US-7.1 — As an Admin, I can add Subcontractors/Vendors to my company with attached insurance certificates and license documents, each with an expiration date.

- Acceptance: The system sends a notification to the Admin 30/14/7 days before an insurance certificate or license expires.

US-7.2 — As an Admin, I can see a company-wide compliance dashboard listing all subcontractors with expiring or expired documentation.

US-7.3 — As a Project Manager, I cannot assign an expired-insurance Subcontractor to an active Project without an explicit Admin override (logged).

- Acceptance: Attempting the assignment surfaces a warning; proceeding requires Admin role and is recorded in the audit log.

Business Rules

- E-signature events (Estimate approval, Change Order approval) are treated as legal compliance records and are retained per the data retention policy in [Security & Compliance](#), Section 7 — they are never deleted, only superseded.

Builders Stream — Technical Architecture

Version: 1.0

Date: 2026-07-07

Related: [PRD](#) · [Functional Requirements](#) · [Database Schema](#) · [NFRs](#)

1. Architecture Pattern

Decoupled Modular Monolith. The frontend and backend are independently deployable services communicating over a versioned HTTP/JSON API. The backend itself is a single deployable service internally organized into strictly-bounded modules (not a microservices mesh) — this avoids distributed-systems overhead while a solo developer builds out the platform, while keeping module boundaries clean enough to extract a module into its own service later if it becomes a scaling bottleneck.

2. Technology Stack

Layer	Technology	Rationale
Frontend	Next.js 16 (App Router), TypeScript, Tailwind CSS	Modern DX, strong ecosystem, SSR for client-facing dashboards.
Backend	FastAPI (Python 3.12)	Async I/O, native OpenAPI schema generation, strong typing via Pydantic v2.
Database	PostgreSQL 16+	Relational integrity, native Row-Level Security, JSONB for flexible fields.
ORM / Migrations	SQLAlchemy (async) + Alembic	Industry-standard Python persistence and migration tooling.
Background Tasks	Celery or Dramatiq + Redis	PDF generation, QuickBooks/FreshBooks sync, notification delivery.
Auth	OIDC/JWT-based (e.g., self-hosted Keycloak or a library such as Better Auth adapted to FastAPI)	Decouples identity from the API; supports future non-web clients.
Payments	Stripe (Billing + Customer Portal)	Handles Builders Stream's own subscription billing (not client-facing project invoicing).
Reverse Proxy / TLS	Traefik or Nginx + Let's Encrypt	Required for self-hosted deployment (Section 8).
Observability	Sentry (errors), PostHog (product analytics)	Self-hostable or cloud, both have generous free/self-host tiers.
API Contract	OpenAPI (auto-generated by FastAPI) → openapi-typescript	Frontend TypeScript types are generated from the backend schema, never hand-written, to prevent drift.

3. Modular Domain Decomposition

The backend is organized as internal packages, one per bounded context:

```
src/
  core/      # tenant context, auth, RBAC, shared utilities
  users/     # Users & Company Management
  crm/       # CRM
  projects/  # Project Management
  estimation/ # Estimation Engine
  billing/   # Accounting & Billing
  integrations/ # QuickBooks / FreshBooks sync
  compliance/ # Subcontractor certs, e-signature records (cross-cutting)
```

Encapsulation rule: a module may only read another module's data through that module's public service interface (a Python function/class), never by querying another module's tables directly. This is enforced by code review discipline (and can later be enforced mechanically with import-linter) rather than by physical service boundaries.

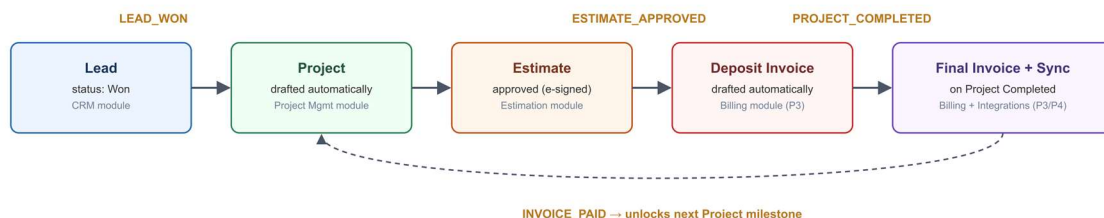
4. Cross-Module Communication: Internal Event Bus

Complex workflows span multiple modules (e.g., CRM lead → Project → Estimate → Invoice). Rather than modules calling each other synchronously and creating tight coupling, state-changing milestones publish events on an in-process (or Redis-backed) event bus:

Event	Published By	Consumed By
LEAD_WON	CRM	Project Management (drafts a new Project)
ESTIMATE_APPROVED	Estimation	Billing (drafts deposit invoice)
PROJECT_COMPLETED	Project Management	Billing (triggers final invoice), Integrations (syncs final job cost to QuickBooks)
INVOICE_PAID	Billing	Project Management (unlocks next milestone, if gated)

This keeps modules loosely coupled: Billing does not need to know CRM's internals, only that an **ESTIMATE_APPROVED** event carries a **project_id** and **approved_total**.

Lead-to-Cash Event Flow



Each arrow is an internal event on the backend's event bus, not a direct cross-module function call.

Lead-to-Cash event flow across modules

5. Multi-Tenancy & Data Isolation

5.1 Tenant Model

- **Shared database, shared schema.** All tenant data lives in the same tables; every tenant-owned table carries a `company_id`.
- **Nested company hierarchy.** `companies.parent_id` self-references, allowing parent/child branch structures (see [Database Schema](#), Section 2).

5.2 Enforcement: PostgreSQL Row-Level Security (RLS)

RLS is the **primary** enforcement mechanism, not just a code-level `WHERE company_id = ...` convention — the database itself rejects cross-tenant access even if application code has a bug.

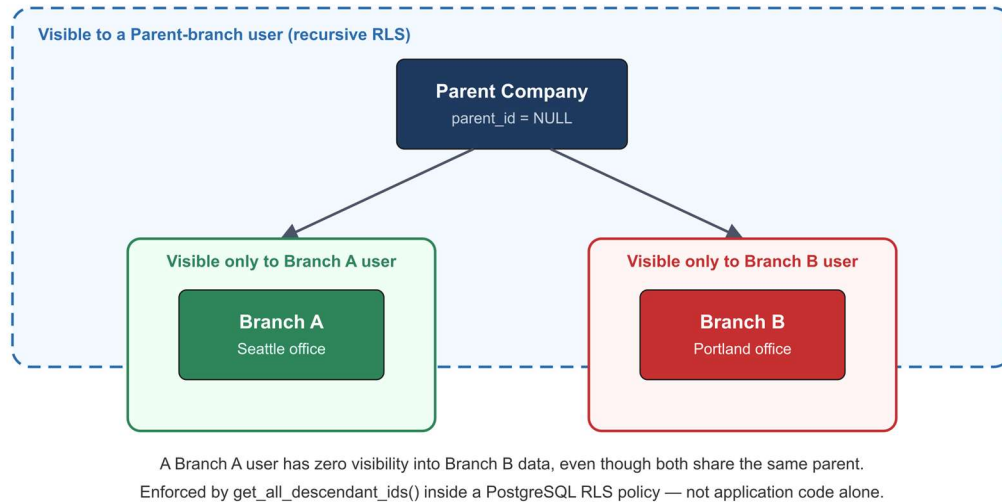
8. A `TenantMiddleware` in FastAPI extracts the active tenant's `company_id` from the validated JWT (or an `X-Tenant-ID` header for a user with access to multiple companies) and stores it in a `contextvars.ContextVar` for the duration of the request.
9. Before any database query executes, the connection issues `SET LOCAL app.current_tenant = '<company_id>'` for that transaction.
10. Every tenant-owned table has an RLS policy such as:

```
```sql
 USING (company_id IN (SELECT
get_all_descendant_ids(current_setting('app.current_tenant')::uuid)))
...
```
```

where `get_all_descendant_ids` is a recursive SQL function that returns the active company plus all of its child branches — giving parent-branch users visibility into child data while strictly isolating siblings from each other.

11. If the tree grows large enough that the recursive lookup becomes a measurable cost, the permitted `company_id` set can be cached in the user's session/JWT at login instead of recomputed per request (see [NFRs](#), Section 2 for the trigger threshold).

Nested Company Hierarchy & Row-Level Security Scope



Nested company hierarchy and RLS visibility scope

5.3 Tenant Context Propagation (Frontend → Backend)

- Frontend attaches **Authorization: Bearer <jwt>** and, for multi-company users, an **X-Tenant-ID** header indicating the "active branch" selected in the UI.
- Backend middleware validates both before any module logic executes; requests without a resolvable tenant context are rejected (except public/auth routes).

6. Estimation Calculation Rules

To guarantee reproducible, auditable quotes, the Estimation module executes calculations server-side, in this fixed order, using Python's **decimal** module (never floats) for all currency math:

12. Line item base cost = **quantity × unit_rate**
13. Category subtotals
14. Overhead markup applied
15. Profit margin applied
16. Tax liability calculated (if applicable)

Client-submitted totals are always ignored in favor of a server-side recompute — the frontend calculation is a preview only.

7. Asynchronous Processing

Operations that are slow or involve third parties run as background tasks (Celery/Dramatiq workers backed by Redis), not inline in the request/response cycle:

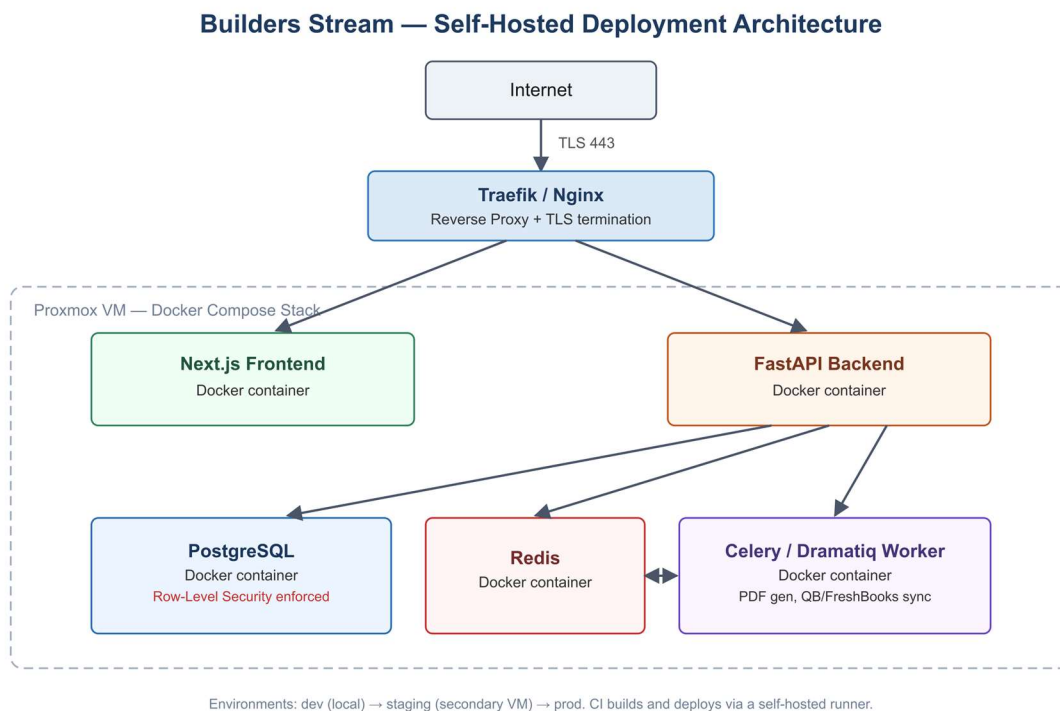
- PDF generation for Estimate proposals (**/estimates/{id}/export**)
- QuickBooks/FreshBooks synchronization

- Compliance expiry notifications (insurance/license reminders)
- Outbound transactional email

The API returns **202 Accepted** immediately for these operations; the frontend polls or receives a websocket/notification event when the job completes.

8. Deployment Topology (Self-Hosted)

Target environment: the developer's own Proxmox/Dell PowerEdge infrastructure, not a managed cloud PaaS.



Self-hosted deployment architecture

- Each component runs as a Docker container, orchestrated via Docker Compose (or a lightweight stack such as Docker Swarm) on a Proxmox VM.
- PostgreSQL data volume and Redis persistence are backed by scheduled, off-host backups (see [NFRs](#), Section 4).
- Environments: **dev** (local), **staging** (a second Proxmox VM or namespace mirroring prod), **prod**.
- CI/CD: a self-hosted runner (e.g., Gitea Actions, Woodpecker CI, or GitHub Actions with a self-hosted runner) builds Docker images on push to **main**, runs the test suite (see [Test Strategy](#)), and deploys to **staging** automatically; promotion to **prod** is a manual, deliberate step.

9. Open Architectural Question: Mobile/Offline

Field crew offline support (Section 7 of the [PRD](#)) is undecided. If required, it will most likely be implemented as a Progressive Web App (PWA) with local-first storage (e.g., IndexedDB) and

background sync against the Project Management module's Daily Log and Task endpoints, rather than a separate native app — this preserves the single-frontend-codebase principle. This is **not** part of the current architecture and would require its own design pass before implementation.

Builders Stream — Database Schema Design

Version: 1.0

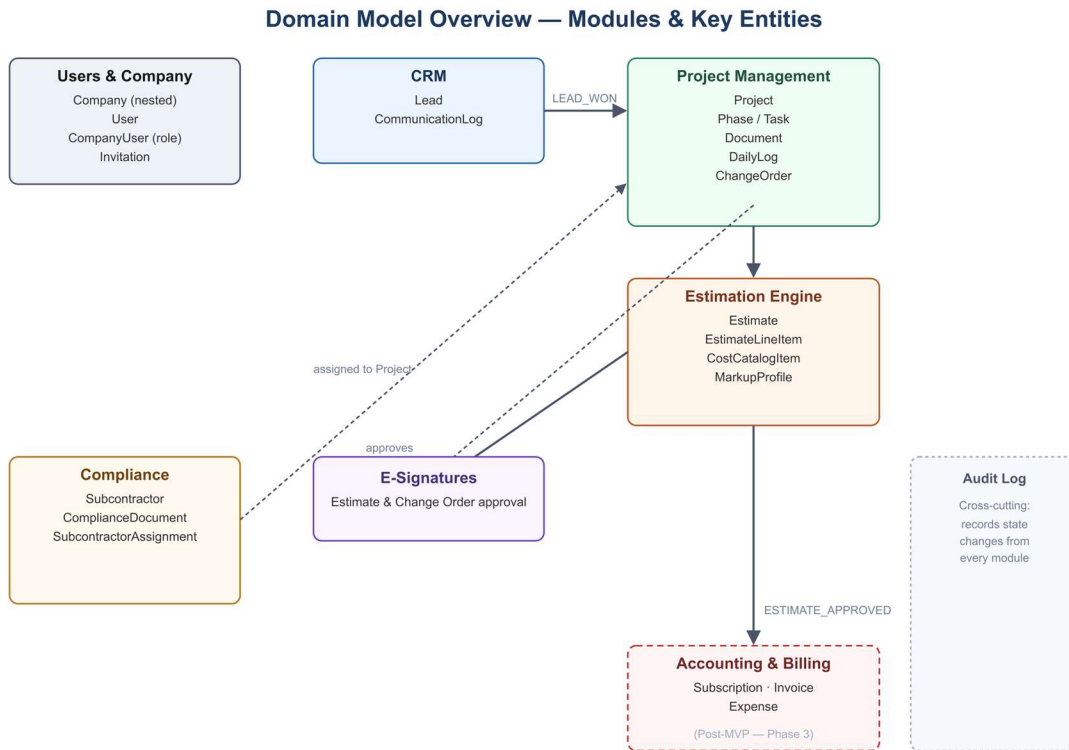
Date: 2026-07-07

Related: [Technical Architecture](#) · [Functional Requirements](#)

All primary keys are `UUID DEFAULT gen_random_uuid()`. All tenant-owned tables carry `company_id UUID NOT NULL REFERENCES companies(id)` and have Row-Level Security enabled per [Technical Architecture](#), Section 5. Timestamps are `TIMESTAMP WITH TIME ZONE`.

Tables below are grouped by module for readability, not by migration order. Some tables reference others defined in a later section (e.g., `change_orders` in Section 4 references `esignatures` from Section 6) — actual Alembic migrations must sequence `CREATE TABLE` statements by foreign-key dependency, not by this document's section order.

1. Entity-Relationship Overview



Domain model overview showing modules and key entities

The diagram above groups entities by module for readability. The full entity-relationship graph, including every foreign key, is expressed precisely in the SQL below (Sections 2–8).

2. Users & Company Management (P0)

```
CREATE TABLE companies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

    parent_id UUID REFERENCES companies(id) ON DELETE CASCADE,
    name VARCHAR(255) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now(),
    is_active BOOLEAN DEFAULT TRUE
);
CREATE INDEX idx_companies_parent_id ON companies(parent_id);

CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    full_name VARCHAR(255),
    created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE company_users (
    company_id UUID REFERENCES companies(id) ON DELETE CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE CASCADE,
    role VARCHAR(50) NOT NULL CHECK (role IN
('admin','project_manager','field_crew','accountant','client')),
    created_at TIMESTAMPTZ DEFAULT now(),
    PRIMARY KEY (company_id, user_id)
);

CREATE TABLE invitations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
    email VARCHAR(255) NOT NULL,
    role VARCHAR(50) NOT NULL,
    expires_at TIMESTAMPTZ NOT NULL,
    accepted_at TIMESTAMPTZ
);

-- Recursive descendant lookup, used by RLS policies across all tenant tables
CREATE OR REPLACE FUNCTION get_all_descendant_ids(company_uuid UUID)
RETURNS TABLE (child_id UUID) AS $$
    WITH RECURSIVE company_tree AS (
        SELECT id FROM companies WHERE id = company_uuid
        UNION ALL
        SELECT c.id FROM companies c INNER JOIN company_tree ct ON c.parent_id = ct.id
    )
    SELECT id FROM company_tree;
$$ LANGUAGE sql STABLE;

ALTER TABLE companies ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON companies
    USING (id IN (SELECT get_all_descendant_ids(current_setting('app.current_tenant')::uuid)));

```

3. CRM (P1)

```

CREATE TABLE leads (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    contact_name VARCHAR(255) NOT NULL,
    project_name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone VARCHAR(20),
    status VARCHAR(20) NOT NULL DEFAULT 'new'
    CHECK (status IN ('new','contacted','estimating','qualified','won','lost')),
    estimated_value NUMERIC(12,2),
    project_type VARCHAR(100) NOT NULL,

```

```

        notes TEXT,
        created_at TIMESTAMPTZ DEFAULT now(),
        updated_at TIMESTAMPTZ DEFAULT now()
    );
CREATE INDEX idx_leads_company_status ON leads(company_id, status);

CREATE TABLE communication_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    lead_id UUID NOT NULL REFERENCES leads(id) ON DELETE CASCADE,
    company_id UUID NOT NULL REFERENCES companies(id),
    author_id UUID NOT NULL REFERENCES users(id),
    channel VARCHAR(20) NOT NULL CHECK (channel IN ('call','email','note','sms')),
    body TEXT NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now()
    -- Immutable by convention: no updated_at, no UPDATE grants at the application layer.
);

ALTER TABLE leads ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON leads
    USING (company_id IN (SELECT
get_all_descendant_ids(current_setting('app.current_tenant')::uuid)));
ALTER TABLE communication_logs ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON communication_logs
    USING (company_id IN (SELECT
get_all_descendant_ids(current_setting('app.current_tenant')::uuid)));

```

4. Project Management (P1)

```

CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    lead_id UUID REFERENCES leads(id),
    name VARCHAR(255) NOT NULL,
    site_address TEXT NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'draft'
    CHECK (status IN
('draft','pre_construction','active','suspended','completed','archived')),
    projected_start_date DATE,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE phases (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    company_id UUID NOT NULL REFERENCES companies(id),
    name VARCHAR(255) NOT NULL,
    sequence INT NOT NULL DEFAULT 0
);

CREATE TABLE tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    phase_id UUID NOT NULL REFERENCES phases(id) ON DELETE CASCADE,
    company_id UUID NOT NULL REFERENCES companies(id),
    name VARCHAR(255) NOT NULL,
    assignee_id UUID REFERENCES users(id),
    due_date DATE,
    status VARCHAR(20) NOT NULL DEFAULT 'open' CHECK (status IN ('open','in_progress','done')),
    created_at TIMESTAMPTZ DEFAULT now()
);

```

```

CREATE TABLE documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  file_name VARCHAR(255) NOT NULL,
  storage_path TEXT NOT NULL,
  version INT NOT NULL DEFAULT 1,
  uploaded_by UUID NOT NULL REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE daily_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  author_id UUID NOT NULL REFERENCES users(id),
  log_date DATE NOT NULL,
  weather VARCHAR(100),
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
  -- Immutable once submitted (application-layer enforced).
);

CREATE TABLE change_orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  description TEXT NOT NULL,
  cost_delta NUMERIC(12,2) NOT NULL,
  schedule_impact_days INT DEFAULT 0,
  status VARCHAR(20) NOT NULL DEFAULT 'pending' CHECK (status IN
('pending', 'approved', 'rejected')),
  signature_id UUID REFERENCES esignatures(id),
  created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS policies follow the same tenant_isolation_policy pattern as Section 3 for every table
above.

```

5. Estimation Engine (P1/P2)

```

CREATE TABLE markup_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  name VARCHAR(255) NOT NULL,
  overhead_pct NUMERIC(5,2) NOT NULL DEFAULT 0,
  profit_pct NUMERIC(5,2) NOT NULL DEFAULT 0
);

CREATE TABLE cost_catalog_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  parent_catalog_item_id UUID REFERENCES cost_catalog_items(id), -- links a branch override to
its parent's item
  category VARCHAR(100) NOT NULL,
  name VARCHAR(255) NOT NULL,
  unit VARCHAR(50) NOT NULL, -- e.g., 'sqft', 'hour', 'each'
  unit_rate NUMERIC(12,2) NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT now()
);

```

```

CREATE TABLE estimates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  project_id UUID REFERENCES projects(id),
  lead_id UUID REFERENCES leads(id),
  markup_profile_id UUID NOT NULL REFERENCES markup_profiles(id),
  status VARCHAR(20) NOT NULL DEFAULT 'draft'
    CHECK (status IN ('draft','sent','approved','rejected')),
  subtotal NUMERIC(12,2),
  total NUMERIC(12,2),
  is_snapshotted BOOLEAN NOT NULL DEFAULT FALSE, -- true once approved; line items become
immutable
  signature_id UUID REFERENCES esignatures(id),
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE estimate_line_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  estimate_id UUID NOT NULL REFERENCES estimates(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  cost_catalog_item_id UUID NOT NULL REFERENCES cost_catalog_items(id),
  quantity NUMERIC(12,2) NOT NULL,
  unit_rate_snapshot NUMERIC(12,2) NOT NULL, -- copied at add-time; immune to later catalog
price changes
  line_total NUMERIC(12,2) NOT NULL
);

```

6. E-Signatures & Compliance (Cross-Cutting)

```

CREATE TABLE esignatures (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  signer_name VARCHAR(255) NOT NULL,
  signer_email VARCHAR(255) NOT NULL,
  signed_at TIMESTAMPTZ NOT NULL,
  ip_address INET NOT NULL,
  signature_artifact_path TEXT NOT NULL, -- rendered signature image/hash, retained per Security
& Compliance doc
  document_type VARCHAR(20) NOT NULL CHECK (document_type IN ('estimate','change_order'))
);

CREATE TABLE subcontractors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  name VARCHAR(255) NOT NULL,
  trade VARCHAR(100),
  contact_email VARCHAR(255),
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE compliance_documents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  subcontractor_id UUID NOT NULL REFERENCES subcontractors(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  doc_type VARCHAR(30) NOT NULL CHECK (doc_type IN ('insurance_certificate','license')),
  storage_path TEXT NOT NULL,
  expires_on DATE NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_compliance_expiry ON compliance_documents(company_id, expires_on);

```

```
CREATE TABLE subcontractor_assignments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  subcontractor_id UUID NOT NULL REFERENCES subcontractors(id),
  company_id UUID NOT NULL REFERENCES companies(id),
  assigned_by UUID NOT NULL REFERENCES users(id),
  override_reason TEXT, -- populated only when assigned despite expired compliance docs (audit trail)
  created_at TIMESTAMPTZ DEFAULT now()
);
```

7. Post-MVP Tables (Phase 3–4, Structure Only)

These are documented now for forward compatibility of foreign keys but are not built until [Roadmap](#) Phase 3–4.

```
CREATE TABLE subscriptions ( -- Builders Stream's own SaaS billing, distinct from client invoicing
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  stripe_customer_id VARCHAR(255) NOT NULL,
  stripe_subscription_id VARCHAR(255) NOT NULL,
  tier VARCHAR(20) NOT NULL CHECK (tier IN ('starter','pro','enterprise')),
  status VARCHAR(20) NOT NULL,
  current_period_end TIMESTAMPTZ
);

CREATE TABLE invoices ( -- client-facing project invoices
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id),
  company_id UUID NOT NULL REFERENCES companies(id),
  amount NUMERIC(12,2) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'draft',
  due_date DATE,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE expenses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  project_id UUID NOT NULL REFERENCES projects(id),
  company_id UUID NOT NULL REFERENCES companies(id),
  description VARCHAR(255) NOT NULL,
  amount NUMERIC(12,2) NOT NULL,
  incurred_on DATE NOT NULL
);

CREATE TABLE integration_connections ( -- QuickBooks / FreshBooks OAuth state
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  provider VARCHAR(20) NOT NULL CHECK (provider IN ('quickbooks','freshbooks')),
  access_token_encrypted TEXT NOT NULL,
  refresh_token_encrypted TEXT NOT NULL,
  connected_at TIMESTAMPTZ DEFAULT now()
);
```

8. Audit Log (Cross-Cutting, P0)

```
CREATE TABLE audit_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

    company_id UUID NOT NULL REFERENCES companies(id),
    actor_id UUID REFERENCES users(id),
    action VARCHAR(100) NOT NULL, -- e.g., 'project.status_changed',
'subcontractor.assigned_with_expired_docs'
    entity_type VARCHAR(50) NOT NULL,
    entity_id UUID NOT NULL,
    metadata JSONB,
    created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_audit_log_company_created ON audit_log(company_id, created_at DESC);

```

9. Indexing & RLS Notes

- Every `company_id` foreign key column should be indexed; most access patterns filter or join on it.
- RLS policies for every table in Sections 3–6 and 8 follow the identical pattern shown in Section 3 — omitted per-table above for brevity, but is a **mandatory** part of each table's migration, not optional.
- `estimate_line_items.unit_rate_snapshot` and `cost_catalog_items.unit_rate` are intentionally separate columns — this is what implements the historical-immutability rule from [Functional Requirements](#), Section 4.

Builders Stream — API Specification

Version: 1.0

Date: 2026-07-07

Related: [Technical Architecture](#) · [Database Schema](#)

This document describes API contracts conceptually. The authoritative, machine-readable contract is FastAPI's auto-generated `openapi.json`, from which frontend TypeScript types are generated (see [Technical Architecture](#), Section 2). Nothing here should be hand-transcribed into frontend types — always regenerate from the live schema.

1. Conventions

- **Base URL:** `https://api.buildersstream.app/v1` (self-hosted equivalent in non-prod environments).
- **Auth:** `Authorization: Bearer <jwt>` on every request except public auth routes.
- **Tenant context:** `X-Tenant-ID: <company_uuid>` required for any user with access to more than one company; optional (inferred from JWT) for single-company users.
- **Pagination:** `?limit=` (default 25, max 100) and `?cursor=` query params; responses include a `next_cursor`.
- **Errors:** JSON body `{ "error": { "code": "string", "message": "string", "details": {...} } }` with standard HTTP status codes (400 validation, 401 unauthenticated, 403 unauthorized/wrong tenant, 404 not found, 409 illegal state transition, 422 unprocessable entity, 429 rate-limited, 5xx server error).
- **Idempotency:** all `POST` endpoints that create billable or externally-synced records accept an optional `Idempotency-Key` header.

2. Users & Company Management

| Route | Method | Purpose | Key Inputs |
|--|--------|---|---------------------------------------|
| <code>/auth/register</code> | POST | Create a new company + first Admin user | Company name, admin email/password |
| <code>/auth/login</code> | POST | Authenticate, issue JWT | Email, password |
| <code>/companies/{id}</code> | GET | Retrieve company detail | — |
| <code>/companies/{id}/children</code> | POST | Create a child branch company | Name, parent implied by path |
| <code>/companies/{id}/users</code> | GET | List users in a company | Pagination |
| <code>/invitations</code> | POST | Invite a user to a company | Email, role |
| <code>/invitations/{token}/accept</code> | POST | Accept an invitation | New user's password (if not existing) |

3. CRM

| Route | Method | Purpose | Key Inputs |
|----------------------------|--------|-----------------------------|--|
| /leads | POST | Create a Lead | company_id, contact_name, project_name, email, phone, project_type |
| /leads | GET | List Leads | Pagination, ?status= filter |
| /leads/{id} | GET | Retrieve a Lead | — |
| /leads/{id} | PATCH | Update Lead fields / status | Any updatable field |
| /leads/{id}/communications | POST | Log a communication | channel, body |
| /leads/{id}/communications | GET | List communication history | Pagination |

4. Project Management

| Route | Method | Purpose | Key Inputs |
|------------------------------|--------|---|--|
| /projects | POST | Initialize a Project | Name, client/lead_id, site_address, projected_start_date |
| /projects | GET | List Projects | Pagination, ?status= filter |
| /projects/{id} | GET | Full project dashboard data | — |
| /projects/{id}/status | PATCH | Advance/revert lifecycle state | New status, optional reason |
| /projects/{id}/phases | POST | Add a Phase | Name, sequence |
| /projects/{id}/tasks | POST | Add a Task | Name, due_date, assignee_id, phase_id |
| /tasks/{id} | PATCH | Update task status/assignment | — |
| /projects/{id}/documents | POST | Upload a Document (new version if filename matches) | File, file_name |
| /projects/{id}/daily-logs | POST | Submit a Daily Log | log_date, weather, notes, photos |
| /projects/{id}/change-orders | POST | Create a Change Order | description, cost_delta, schedule_impact_days |

| Route | Method | Purpose | Key Inputs |
|---|--------|---------------------------------|--------------|
| <code>/change-orders/{id}/send-for-signature</code> | POST | Route to client for e-signature | signer_email |

5. Estimation Engine

| Route | Method | Purpose | Key Inputs |
|---|--------|---|---|
| <code>/catalogs/items</code> | GET | List cost catalog items for active branch | Pagination, category filter, search |
| <code>/catalogs/items</code> | POST | Create/override a catalog item | category, name, unit, unit_rate |
| <code>/markup-profiles</code> | POST | Create a Markup Profile | name, overhead_pct, profit_pct |
| <code>/estimates</code> | POST | Initialize an Estimate | project_id or lead_id, markup_profile_id |
| <code>/estimates/{id}/lines</code> | PUT | Batch replace line items | Array of <code>{cost_catalog_item_id, quantity}</code> |
| <code>/estimates/{id}/calculate</code> | POST | Server-side recalculation | — (uses saved state) |
| <code>/estimates/{id}/export</code> | POST | Queue branded PDF generation | Target email, template_id → returns <code>202 Accepted</code> |
| <code>/estimates/{id}/send-for-signature</code> | POST | Route to client for e-signature | signer_email |
| <code>/esignatures/{id}</code> | GET | Retrieve signature record (audit) | — |

6. Accounting & Billing (Post-MVP)

| Route | Method | Purpose | Key Inputs |
|--|--------|---|----------------------------------|
| <code>/subscriptions/me</code> | GET | Current company's Builders Stream plan | — |
| <code>/subscriptions/portal-session</code> | POST | Create a Stripe Customer Portal session | — |
| <code>/projects/{id}/invoices</code> | POST | Create a client-facing invoice | amount, due_date |
| <code>/projects/{id}/expenses</code> | POST | Record an expense | description, amount, incurred_on |

| Route | Method | Purpose | Key Inputs |
|--|--------|---|------------|
| /reports/profitability | GET | Company-wide project profitability report | Date range |

7. External Integrations (Post-MVP)

| Route | Method | Purpose | Key Inputs |
|--|--------|-------------------------|---------------|
| /integrations/quickbooks/connect | GET | Begin OAuth flow | — (redirects) |
| /integrations/quickbooks/callback | GET | OAuth callback | code, state |
| /integrations/{provider}/sync-status | GET | Last sync status/errors | — |

8. Compliance

| Route | Method | Purpose | Key Inputs |
|---|--------|--|---|
| /subcontractors | POST | Add a Subcontractor/Vendor | name, trade, contact_email |
| /subcontractors/{id}/compliance-documents | POST | Attach an insurance cert or license | doc_type, file, expires_on |
| /compliance/dashboard | GET | Company-wide expiring/expired doc list | — |
| /projects/{id}/subcontractor-assignments | POST | Assign a subcontractor to a project | subcontractor_id, override_reason (if compliance expired) |

9. Webhooks (Inbound)

| Route | Source | Purpose |
|---|--|--|
| /webhooks/stripe | Stripe | Subscription lifecycle events (payment succeeded/failed, plan changed) |
| /webhooks/esignature-provider | E-signature provider (if a third-party service is used instead of a built-in flow) | Signature completed/declined |

10. Security Requirements on Every Endpoint

- Every route (except [/auth/*](#) and public webhook receivers) requires a valid JWT resolving to an active [company_users](#) membership with sufficient role, enforced by a

FastAPI dependency, in addition to the database-level RLS policy (defense in depth — see [Security & Compliance](#)).

- Webhook receivers verify provider signatures (e.g., Stripe's **Stripe-Signature** header) before processing.

Builders Stream — Non-Functional Requirements

Version: 1.0

Date: 2026-07-07

Related: [Technical Architecture](#) · [Security & Compliance](#)

1. Performance

| Requirement | Target |
|---|---|
| API response time (p95, non-report endpoints) | < 300ms under normal load |
| Estimate recalculation
(/estimates/{id}/calculate) | < 1s for estimates up to 200 line items |
| PDF proposal generation | Asynchronous; queued job completes within 30s |
| Dashboard initial load (frontend) | < 2s time-to-interactive on broadband |

2. Scalability

- Must support **1,000+ company tenants** on shared-schema PostgreSQL without per-tenant infrastructure.
- Indexing strategy (see [Database Schema](#), Section 9) must keep core list/detail queries performant as row counts grow into the millions.
- **Trigger for revisiting the recursive RLS lookup:** if `EXPLAIN ANALYZE` on `get_all_descendant_ids` shows measurable latency (baseline: establish this once real company-tree depth/fan-out data exists — do not pre-optimize before that), cache the permitted tenant-ID set in the session/JWT instead of recomputing per request.
- **Trigger for extracting a module into its own service:** a module's background job queue depth or CPU usage disproportionately dominates the shared backend's resource budget, as observed via the Section 5 monitoring below — not before.

3. Availability

- Target **99.5% uptime** for the self-hosted production environment (roughly 3.6 hours of downtime/month budget) — a realistic target for a single-operator self-hosted deployment, not a marketing claim.
- Planned maintenance windows are acceptable and should be communicated in-app in advance; they do not count against the target if scheduled outside business hours and announced.
- No multi-region failover in v1 — this is a single self-hosted environment. Documented as a known limitation, not a gap to silently ignore.

4. Backup & Disaster Recovery

- PostgreSQL: automated daily full backups + continuous WAL archiving, retained 30 days, stored off the Proxmox host (a separate physical or cloud location).
- Document/file storage: mirrored to a secondary location on the same backup cadence.
- Recovery Point Objective (RPO): ≤ 24 hours (bounded by backup cadence; WAL archiving can reduce this further once implemented).
- Recovery Time Objective (RTO): documented and tested via an actual restore drill before the platform accepts its first paying subscriber — not assumed to work untested.

5. Observability

- **Error tracking:** Sentry (self-hosted or cloud) on both frontend and backend.
- **Product analytics:** PostHog for feature usage, funnel tracking (Lead → Won → Estimate Approved).
- **Infrastructure monitoring:** container health, CPU/memory/disk on the Proxmox host, PostgreSQL connection pool saturation, Redis/Celery queue depth — minimum viable setup is Prometheus + Grafana or an equivalent self-hosted stack, since there is no managed-cloud dashboard to rely on.
- **Alerting:** on-call notification (email/SMS/push) for: service down, backup failure, disk usage above 85%, queue depth exceeding a defined threshold.

6. Self-Hosted Infrastructure Requirements

- Minimum viable production footprint: one Proxmox VM (or set of VMs) capable of running the Docker Compose stack in [Technical Architecture](#), Section 8 — exact CPU/RAM/storage sizing should be benchmarked against real load rather than assumed, given the existing Dell PowerEdge T620 hardware's known specs are not detailed in this document.
- Network: static IP or dynamic DNS, valid TLS certificate (Let's Encrypt via the reverse proxy), firewall rules restricting direct database/Redis access to the internal Docker network only.
- Staging environment mirrors production topology at smaller scale, used for pre-deployment verification (see [Test Strategy](#)).

7. Browser & Device Support

- Modern evergreen browsers (Chrome, Edge, Safari, Firefox — current and prior major version).
- Responsive layout down to tablet width (768px) as a baseline; phone-width usability is best-effort in v1 pending the mobile/offline decision noted in [PRD](#), Section 7.

8. Data Retention

- See [Security & Compliance](#), Section 7 for retention rules specific to e-signature and audit-log records, which have longer/immutable retention requirements than general operational data.

Builders Stream — Security & Compliance Plan

Version: 1.0

Date: 2026-07-07

Related: [Technical Architecture](#) · [Functional Requirements](#), Section 7

1. Authentication

- OIDC/JWT-based session management. Passwords hashed with a modern, salted algorithm (e.g., Argon2id) — never reversible encryption.
- JWTs are short-lived (e.g., 15 minutes) with a refresh-token rotation flow; refresh tokens are revocable server-side (e.g., on logout, password change, or suspected compromise).
- Multi-factor authentication (TOTP) is a requirement for the Admin role at minimum, strongly recommended for all roles — should be scoped into an early phase given the platform handles client financial/contract data.

2. Authorization (RBAC Matrix)

| Module | Admin | Project Manager | Field Crew | Accountant | Client |
|--------------------|-----------|---------------------------------------|-----------------------------------|------------------------------|--------------------------------------|
| Users & Company | Full CRUD | Read own company | — | — | — |
| CRM | Full CRUD | Full CRUD | — | — | — |
| Project Management | Full CRUD | Full CRUD | Read assigned + create Daily Logs | Read (financial fields only) | Read sanitized dashboard |
| Estimation | Full CRUD | Full CRUD | — | Read | Approve/reject own estimate (e-sign) |
| Accounting/Billing | Full CRUD | — | — | Full CRUD | Read own invoices |
| Compliance | Full CRUD | Read + assign (with override logging) | — | Read | — |

Enforced at two layers, per [Technical Architecture](#), Section 5: a FastAPI dependency checks role before the request reaches business logic (fast-fail, clear error), and PostgreSQL RLS enforces tenant boundary regardless of application-layer bugs (defense in depth).

3. Tenant Isolation

- Row-Level Security is the authoritative isolation mechanism (see [Technical Architecture](#), Section 5) — application-layer `WHERE company_id = ...` filtering is a performance/clarity convenience, never the sole safeguard.
- Automated isolation tests are a **release gate**, not optional: no deploy to staging/prod proceeds if the RLS test suite (see [Test Strategy](#), Section 2) fails.

4. Encryption

- **In transit:** TLS 1.2+ enforced at the reverse proxy for all external traffic; internal Docker-network traffic between backend/DB/Redis is not exposed externally.
- **At rest:** PostgreSQL data volume encrypted at the disk/filesystem level (e.g., LUKS on the Proxmox host); third-party OAuth tokens (QuickBooks/FreshBooks — see [Database Schema](#), Section 7) are additionally encrypted at the application layer before storage, never stored in plaintext.
- **Backups:** encrypted at rest in their off-host storage location.

5. Audit Logging

- Every state-changing action on financially or legally significant entities (Project status changes, Estimate approval, Change Order approval, Subcontractor compliance overrides, user role changes) writes an `audit_log` row (see [Database Schema](#), Section 8) recording who, what, when, and relevant metadata.
- Audit log entries are append-only; no application code path updates or deletes them.

6. E-Signature Workflow

Builders Stream implements a first-party e-signature capture for Estimates and Change Orders (see [Functional Requirements](#), US-4.5 and US-3.6), rather than assuming a third-party e-sign service — the architecture doesn't preclude integrating one later (Section 9 of the [API Specification](#) reserves a webhook route for that).

To satisfy the intent-to-sign standard under the U.S. E-SIGN Act (and equivalent state UETA statutes):

- Capture: signer's typed/drawn signature, full name, email, timestamp, and originating IP address (`esignatures` table, [Database Schema](#), Section 6).
- Present the signer with the exact document version being signed (the snapshotted Estimate or Change Order), not a live/mutable one.
- Retain the signed artifact and its metadata indefinitely as a legal record (see Section 7 below) — it must never be deleted, even if the underlying Project or company is later deactivated.
- **This is not a substitute for legal review.** Before this workflow is relied upon for binding contracts, it should be reviewed by counsel familiar with construction contract law in the jurisdictions Builders Stream's subscribers operate in — this document specifies the technical capture requirements, not a legal opinion on enforceability.

7. Data Retention

| Record Type | Retention |
|--|--|
| E-signature records
(esignatures) | Indefinite / immutable |
| Audit log | Minimum 7 years (typical construction-contract statute-of-limitations window; confirm against subscribers' jurisdictions) |
| Compliance documents
(insurance/license) | Retained for the life of the subcontractor relationship + 7 years |
| General operational data (tasks, daily logs, communications) | Retained for the life of the tenant account; purged within 90 days of a company's confirmed account deletion request, except where audit/e-signature retention above overrides |
| Deactivated company data | Not deleted on deactivation (is_active = false) — only on explicit, confirmed deletion request |

8. Compliance Tracking Feature (Insurance/License Expiry)

- Implements [Functional Requirements](#) US-7.1–US-7.3: notification cadence (30/14/7 days before expiry), compliance dashboard, and mandatory Admin-logged override to assign a subcontractor with expired documentation.
- This is a workflow/notification feature within Builders Stream, not a certification authority — the platform does not verify the authenticity of uploaded insurance certificates or licenses; that remains the subscribing company's responsibility. State this limitation clearly in-product.

9. Incident Response (Outline)

- Defined escalation path for a suspected data breach or tenant-isolation failure: immediate containment (revoke affected credentials/tokens), root-cause investigation, affected-tenant notification per applicable state breach-notification law, and a post-incident review.
- Given the solo-developer/self-hosted context, this is deliberately lightweight in v1 but must exist in writing (a runbook) before the platform holds real subscriber financial/contract data — not deferred indefinitely.

10. Third-Party Data Processors

Any subprocessor handling subscriber data (Stripe, email delivery provider, error tracking, QuickBooks/FreshBooks) must be listed in Builders Stream's Terms of Service / Privacy Policy (a legal deliverable outside the scope of this technical document, but its accuracy depends on this Security & Compliance Plan staying current).

Builders Stream — Pricing & Subscription Model

Version: 1.0

Date: 2026-07-07

Related: [PRD](#) · [Functional Requirements](#), Section 5

1. Model: Tiered Plans by Feature + Seat Count

Three tiers gate access by **module availability** and scale cost by **active user seats**. This rewards companies that grow their team while keeping the entry tier affordable for a small renovation shop.

2. Tier Definitions

| Tier | Target Company Size | Modules Included | Seat Model |
|-------------------|---|--|---|
| Starter | Solo contractor / very small crew | Users & Company, CRM, Project Management | Included seats: small fixed number; overage priced per seat |
| Pro | Small/medium company (the primary target for the 1,000-subscriber goal) | Everything in Starter + Estimation Engine + Compliance Tracking | Included seats: moderate; overage priced per seat |
| Enterprise | Multi-branch companies using the nested company hierarchy | Everything in Pro + Accounting/Billing + QuickBooks/FreshBooks Integration + nested child-branch support | Custom seat pricing; volume discount |

Note on exact dollar amounts: this document intentionally does not invent per-tier prices. Pricing should be set from real market research (competitor pricing, willingness-to-pay interviews with target contractors) before launch, and is tracked as [PRD](#) Open Question #2. The structure above (what's gated by tier) is the design decision this document commits to; the numbers are a pending business decision, not a technical one.

3. Feature Gating Table

| Module | Starter | Pro | Enterprise |
|---------------------------------------|---------|-----|------------|
| Users & Company Management | ✓ | ✓ | ✓ |
| CRM | ✓ | ✓ | ✓ |
| Project Management | ✓ | ✓ | ✓ |
| Estimation Engine (incl. e-signature) | — | ✓ | ✓ |
| Compliance Tracking | — | ✓ | ✓ |

| Module | Starter | Pro | Enterprise |
|-------------------------------------|---------|--------------------|------------|
| Accounting & Billing | — | — | ✓ |
| QuickBooks / FreshBooks Integration | — | — | ✓ |
| Nested child-branch companies | — | Single branch only | ✓ |

Enforcement happens server-side: the `TenantMiddleware` (see [Technical Architecture](#), Section 5) resolves the active company's `subscriptions.tier` and the relevant module's FastAPI dependency rejects requests with `403` if the tier doesn't include that module — never enforced only in the frontend UI.

4. Stripe Implementation Mapping

- Each tier is a Stripe **Product** with a **Price** (monthly and annual variants).
- Seat overage is modeled as a Stripe **metered/usage-based** line item on the subscription, incremented when a company's active user count exceeds the tier's included seats.
- `subscriptions` table (see [Database Schema](#), Section 7) stores `stripe_customer_id`, `stripe_subscription_id`, `tier`, `status`, and `current_period_end`, kept in sync via the `/webhooks/stripe` endpoint ([API Specification](#), Section 9).
- Plan changes (upgrade/downgrade) and cancellations are handled through Stripe's Customer Portal rather than custom UI, per the original architectural decision to avoid building billing logic from scratch.

5. Trial & Conversion

- New companies start on a time-boxed trial (e.g., 14 days) with **Pro**-tier feature access, to let a prospective subscriber experience the Estimation Engine — the feature most likely to demonstrate value over their current spreadsheet-based workflow.
- On trial expiry without a payment method, the account downgrades to a read-only state (data preserved, write access blocked) rather than being deleted — consistent with the data retention policy in [Security & Compliance](#), Section 7.

6. Upgrade / Downgrade Rules

- Upgrades take effect immediately (proration handled by Stripe).
- Downgrades that would remove access to a module currently in use (e.g., Enterprise → Pro while QuickBooks integration is active) surface a confirmation warning listing exactly what will become inaccessible, and take effect at the end of the current billing period rather than immediately.

Builders Stream — Roadmap & Implementation Plan

Version: 1.0

Date: 2026-07-07

Related: [PRD](#) · [Functional Requirements](#)

Pacing assumption: solo developer, no fixed external deadline. Phases are ordered by **dependency**, not by calendar — each phase should be substantially complete (including its tests, per [Test Strategy](#)) before the next begins, since later modules assume earlier ones exist and work correctly.

Implementation Roadmap



Phases are ordered by dependency, not calendar — each completes (with tests) before the next begins.

Implementation roadmap phases 0 through 5

Phase 0 — Foundation (Pre-MVP)

Goal: nothing else can be built safely without this.

- Dockerized dev environment (PostgreSQL, Redis, backend, frontend containers) per [Technical Architecture](#), Section 8.
- `companies / users / company_users` schema + nested-hierarchy `parent_id` + `get_all_descendant_ids()` function.
- RLS policies enabled and proven on the Users & Company tables.
- `TenantMiddleware`: JWT validation → `contextvars` → `SET LOCAL app.current_tenant`.
- Auth: registration, login, invitations.
- Audit log table and a working write path.
- **Exit criteria:** automated RLS isolation tests (two companies, verify zero cross-visibility) pass in CI.

Phase 1 — CRM & Project Management (MVP core)

- CRM: Lead CRUD, pipeline status transitions, communication logs.
- Project Management: Project CRUD/lifecycle state machine, Phases, Tasks, Documents, Daily Logs.
- `LEAD_WON` → draft Project event wiring (see [Technical Architecture](#), Section 4).
- Client-facing read-only project dashboard (sanitized view, RBAC-scoped).

- **Exit criteria:** a Lead can be created, moved to Won, and land as a Draft Project with client details carried over, end-to-end, with tests.

Phase 2 — Estimation Engine + E-Signature (MVP completion)

- Cost Catalog, Markup Profiles (with parent/child override inheritance).
- Estimate creation, line items, server-side calculation pipeline (fixed-point decimal, fixed order of operations).
- PDF export as an async job.
- E-signature capture flow for Estimate approval (`esignatures` table, [Security & Compliance](#), Section 6).
- Change Order creation + e-signature approval on active Projects (reuses the e-signature capability built for Estimates).
- Historical immutability / snapshotting on approval.
- `ESTIMATE_APPROVED` event published (consumed starting in Phase 3).
- **Exit criteria: this is the MVP launch bar.** Users/Company + CRM + Project Management + Estimation Engine (including e-signature) are feature-complete, tested, and deployed to production. This matches the MVP scope defined in [PRD](#), Section 6.

Phase 3 — Compliance Tracking + Accounting/Billing (Post-MVP)

Grouped together because both are needed before Enterprise-tier subscribers can be onboarded (see [Pricing Model](#), Section 3), and Billing's invoice generation is the natural consumer of the `ESTIMATE_APPROVED` event from Phase 2.

- Subcontractor/Vendor records, compliance document upload + expiry notifications, compliance dashboard, assignment override + audit logging.
- Builders Stream's own Stripe subscription billing (tiers, seats, Customer Portal, webhook sync).
- Client-facing Project invoicing and Expense tracking.
- Profitability reporting.
- **Exit criteria:** a company can subscribe/pay via Stripe, and an approved Estimate flows into a draft client invoice automatically.

Phase 4 — External Integrations (Post-MVP)

- QuickBooks OAuth connect + async sync of invoices/expenses.
- FreshBooks OAuth connect + async sync (same pattern).
- Sync status visibility and retry-on-failure handling.
- **Exit criteria:** Enterprise-tier companies can connect and see a successful, monitored sync.

Phase 5 — Open Items (Scope TBD, Not Yet Scheduled)

These are explicitly deferred pending decisions noted in [PRD](#), Section 8, and are **not** part of any committed phase above:

- Offline-capable mobile/PWA support for field crews.

- AI-assisted blueprint takeoff for the Estimation Engine.
- Multi-currency / multi-language support.

Milestone Summary

| Phase | Delivers | Depends On |
|-------|--|------------|
| 0 | Multi-tenant foundation, auth, RLS | — |
| 1 | CRM + Project Management | Phase 0 |
| 2 | Estimation + e-signature — MVP launch | Phase 1 |
| 3 | Compliance + Billing | Phase 2 |
| 4 | QuickBooks/FreshBooks | Phase 3 |
| 5 | Open items (unscheduled) | Varies |

Builders Stream — Test Strategy

Version: 1.0

Date: 2026-07-07

Related: [Technical Architecture](#) · [Security & Compliance](#) · [Roadmap](#)

1. Test Pyramid

| Layer | Scope | Tooling |
|------------------|---|---|
| Unit | Estimation calculation engine, state machines (Project/Lead/Estimate status), Pydantic validation | pytest |
| Integration | API contract per module, database RLS enforcement, event bus wiring | pytest + a real PostgreSQL test database (not mocked) |
| End-to-End | Full user flows through the Next.js frontend against a running backend | Playwright |
| Load/Performance | Multi-tenant query performance at scale | k6 or Locust against a seeded staging dataset |

Business logic (estimation math, state transitions, RLS enforcement) is verified against a real database and real calculations — not mocked — per this project's standing practice of testing against real dependencies rather than mocks that can silently drift from production behavior.

2. Tenant Isolation Testing (Release Gate)

This is the single most important test category given the shared-schema multi-tenancy model ([Technical Architecture](#), Section 5) — a failure here is a data breach, not a bug.

Required test cases, run in CI on every merge to `main`:

17. Seed two unrelated companies (A and B) each with Leads, Projects, and Estimates.
18. Authenticate as a user in Company A; attempt to **GET/PATCH/DELETE** every entity type by Company B's IDs directly (not just list endpoints — direct ID access is the common bypass). Assert **404/403** on every attempt, never a **200** with B's data.
19. Seed a parent company P with child branches C1 and C2. Authenticate as a P-level user; assert visibility into both C1 and C2's data. Authenticate as a C1 user; assert **no** visibility into C2's data (siblings must stay isolated even though both share a parent).
20. Deliberately omit the **X-Tenant-ID** header / use a malformed one; assert the request is rejected rather than silently defaulting to some other tenant.
21. A regression test that runs a query with RLS temporarily disabled (in test setup only) to confirm the policy itself — not just application code — is what blocks cross-tenant access; this catches the case where a policy is accidentally dropped or misconfigured in a migration.

This suite failing blocks deployment to staging and production, per [Security & Compliance](#), Section 3.

3. Estimation Engine Testing

- Verify the fixed calculation order (quantity × rate → category subtotal → overhead → profit → tax) against hand-computed expected values for representative estimates.
- Verify all monetary math uses `decimal`, not `float` — a test that would catch a regression back to floating-point rounding errors.
- Verify historical immutability: change a `cost_catalog_items.unit_rate` after an Estimate is approved and assert the approved Estimate's totals are unchanged.
- Verify parent/child catalog override precedence.

4. State Machine Testing

- For Project, Lead, and Estimate status fields: table-driven tests asserting every *legal* transition succeeds and every *illegal* transition (e.g., Project Draft → Completed) is rejected with `409`.

5. E-Signature & Compliance Testing

- Verify an Estimate/Change Order cannot be approved without a captured signer name, email, timestamp, and IP.
- Verify the exact document version presented to the signer matches the snapshotted version stored (no live-document substitution bug).
- Verify a Subcontractor with an expired compliance document cannot be assigned to a Project without an Admin role and a recorded `override_reason`, and that the attempt is written to `audit_log`.

6. Third-Party Integration Testing

- **Stripe:** test-mode API keys and Stripe CLI's webhook forwarding for local/staging testing of subscription lifecycle events; verify idempotent webhook handling (Stripe can redeliver events).
- **QuickBooks/FreshBooks:** sandbox accounts for both providers; verify sync retries on transient failure and that a sync failure never blocks the originating Builders Stream action (invoice/expense creation succeeds even if the downstream sync later fails and is retried).

7. Performance/Load Testing

- Before the 1,000-subscriber target, run a load test against a seeded dataset simulating that scale (companies, nested branches, leads, projects, estimates) to validate the performance targets in [NFRs](#), Section 1, and to establish the actual baseline for the RLS-caching trigger in [NFRs](#), Section 2 — not skipped in favor of assuming it'll be fine.

8. CI Gates

A change cannot merge to `main` unless:

22. Unit + integration test suites pass.
23. Tenant isolation suite (Section 2) passes.

24. Linting/type-checking passes (`mypy/ruff` for backend, `tsc/eslint` for frontend).
25. OpenAPI schema diff is reviewed when backend contracts change, to catch accidental breaking changes to the frontend's generated types (see [Technical Architecture](#), Section 2).

E2E (Playwright) and load tests run on a scheduled/pre-release basis rather than on every commit, given their runtime cost, but are mandatory before promoting a Phase's exit criteria (see [Roadmap](#)) to production.